

GHOSTBRIDGE × NETMAKER

Netmaker isn't a dependency in this project.

It's core code.

A zero-trust, compliance-grade AI control plane — built in Rust, running on a Netmaker mesh that we treat as the spine: the transport **and** the root of identity.

Rust · 31-crate workspace

No-NIC hardened mesh

OSCAL-gated

Tamper-evident chain

Confrontable AI

Honest note: this is the first time anyone outside the project will have seen it. We booked the trial call because it was the door — but what we really want is for a sharp engineer on your side to look closely and tell us, straight, what you think.

● cognitive-mcp verified live this session

THE ONE PARAGRAPH

A mesh you can **trust**, not just connect

Traffic flows through a hardened pipeline — decoy ingress, identity sled, Xray header injection, a per-hop interceptor — where every request is classified, routed, gated against machine-readable policy (OSCAL), and recorded into a tamper-evident hash-linked chain. An AI layer reasons over that verified history but **cannot execute**

anything itself. Same codebase ships as a **privacy** edition or a **compliance** edition — one config flag decides which world you're in.

HOW IT WORKS

One governance pattern, every layer

Make the wrong thing impossible first; gate what's left; record everything. The same idea at three scales — **Rust** says code can't be wrong, **schema** says state can't be invalid, **A.N.N.A.** says action can't be unauthorized — and the chain says it's all on the record.

RUST

the compiler is the first enforcer – invalid code won't build



SCHEMA

invalid STATE is unrepresentable – the silent gate, before any rule runs



TRAFFIC COP · Gemma + Xray

Gemma classifies (what is this?) + Xray routes (where to?) – directs, never authorizes



ZEROCLAW

routes to the right MODEL by cost / reasoning / specialty – schema-native, can't route off-policy



A.N.N.A.

OSCAL-gated approval at EVERY hop (HMAC-stamped identity header) – the higher-up that says yes/no



CHAIN

hash-linked, tamper-evident evidence – every action becomes a block



THE LOOP

AI reassembles chain → Qdrant (semantic) + Cozo (realtime graph) → reasons → new blocks

WHAT'S ON EVERY BLOCK

Identity vector × Mutation vector

Two structured vectors ride every entry in the chain. Together they make "who changed what, when, on whose authority" a provable query — and they're exactly what A.N.N.A. gates on.

- **Identity vector — the WHO**

```
session_id = Argon2(PSK, salt=WG_pubkey)
```

Derived from the Netmaker WireGuard key. Stable, deterministic, unforgeable. Privacy mode: the key alone (anonymous, consistent). Compliance mode: resolves to a verified user.

- **Mutation vector — the WHAT-CHANGED**

The **mut** class of the subid taxonomy: *what kind of change, on what resource, under what OSCAL authority*. Reads (obs) and changes (mut) are distinct classes — so you can prove nothing was altered, not just that it was logged.



Cross-examination becomes exact: identity #abc performed mut **config.write** on **netmaker.peer** at T, authorized by OSCAL **authorizing-official**. That's the receipt.

THE SEMANTIC MEMORY

Voyage vectors → searchable meaning

Identity & mutation vectors are *structured provenance*. **Voyage vectors are meaning**. Every block, document, and transcript is embedded by **voyage-code-3** into **Qdrant** — so the chain

isn't just verifiable, it's *searchable by similarity*. This is the bottom pane when you cross-examine the agent, and the raw material the loop reasons over.

● Embed

voyage-code-3 turns each chunk of the chain (actions, configs, docs, transcripts) into a high-dimensional vector. Code-aware, so it understands infra, not just prose.

● Store & search

Vectors land in Qdrant, keyed `notebook:source:chunk`. Ask *"show me peers like the one that misbehaved"* — similarity, not keywords.

● Rebuildable

Qdrant is a **derived projection** of the chain. Lose it in a disk casualty → re-embed from the canonical blocks. Trust lives in the chain; retrieval lives here.



Two vector families, two jobs: **Voyage** = "find things *like* this" (semantic). **Identity / mutation** = "who did *exactly* this, and was it allowed" (provenance). The AI uses both — meaning to reason, provenance to prove.

WHY IT'S DIFFERENT

Four things nobody else can credibly claim

● A brain with no hands

The AI is a **delegator, recommender, fixer** — and that's the ceiling. **No internet, no execution rights, no execution control.** It proposes into a schema-shaped mold; a separate, OSCAL-gated, chained layer disposes. A compromised model can't do damage — worst case is a rejected suggestion.

- **You can cross-examine it**

Every action is a chained, searchable block. Confront the agent in real time — "**why did you do this?**" — and it answers with cryptographic receipts pulled live by semantic search. If the model confabulates, the evidence pane contradicts it. **An AI you can put on the stand.**

- **The chain is the truth**

The hash-linked chain is canonical and tamper-evident. Qdrant + Cozo are **AI-rebuilt projections** of it — lose them, re-derive them. It's a blockchain *by structure*, kept honest by **anchoring**, not consensus — the right call for a single-operator compliance ledger.

- **Schema-native, all the way down**

Components were **chosen because they're schema-based from the ground up** (e.g. zeroclaw), and the code is safe by construction (Rust — memory-safe, race-free, single static binaries). Governance holds at every layer because nothing was bolted on.

ONE DIAL

As **Snowden** as you want — or as **Big Brother** as you want

Same mesh, same chain, same confrontable AI. **Only the identity-linkage layer changes**, and it's a deploy-time config — not a rewrite, not a hidden switch.

PRIVACY EDITION

COMPLIANCE EDITION



key IS identity
no PII · no email
de-anon cryptographically impossible

identity .. verified user
vault escrow + user graph
de-anon only on signed OSCAL authority

Even in full-oversight mode, de-anonymization isn't a backdoor — it requires a signed OSCAL authorizing-official action that **itself becomes a chained, confrontable block**. You can cross-examine the watchers.

FOR NETMAKER SPECIFICALLY

A control server with **no network interface**

The `netmaker`, `netmaker-mq` (Mosquitto) and `netmaker-ui` containers run loopback-only. Exposed to the host as unix sockets via Incus proxy devices; `wg-xray` + `Xray` terminate TLS and route by SNI to those sockets. **A port scan of the Netmaker server returns nothing.**



USE CASES

Three editions, one engine

■ Privacy

Where the key *is* the identity and de-anonymization is impossible by design.

- Journalist / source networks — no PII to subpoena
- Activist & at-risk community meshes
- Privacy-first SaaS where you *can't* hand over what you never linked
- Confidential M&A / deal rooms
- Personal sovereign infrastructure

■ EU Compliance

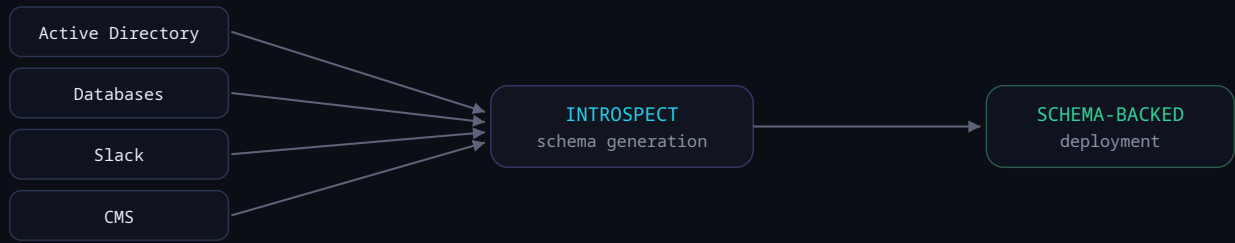
OSCAL-mapped, attributable, auditor-grade — with privacy still defensible.

- GDPR — lawful basis + data-minimization enforced at the schema, not promised
- NIS2 — per-hop enforcement + tamper-evident incident evidence
- DORA — ICT risk & audit trail for financial entities
- EU AI Act — a cross-examinable, no-execution AI with full decision provenance
- Data residency — sovereign mesh, no silent egress

■ Enterprise drop-in

Point it at what you already run. It **introspects and generates a schema-backed deployment** — no greenfield.

- **Active Directory** → groups become OSCAL roles
- **Databases** → schemas introspected into governed state
- **Slack** → workspaces/channels mapped as principals & routes
- **CMS** → content + access model wrapped under the chain
- Result: an existing org, now zero-trust + audited, in days not quarters



The same schema-driven engine that enforces also *renders* — so introspecting your stack produces governance, a control panel, and audit, from one source.

WHY NETMAKER IS CORE

You're already load-bearing

- **First-class state plugin**

Managed by the same engine that runs the whole platform — not wrapped at the edge.

- **Peers are the identity**

`session_id = Argon2(PSK, salt=WG_pubkey)` — rooted in the WireGuard key you already manage. Remove Netmaker and identity has no root. It's **link #0 of the chain**.

- **We expand your surface**

Every GhostBridge deployment is a hardened Netmaker deployment, sold into rooms Netmaker alone doesn't reach yet.

WHY WE'RE SENDING THIS

Not an ask. A **heads-up**.

We're not asking you for anything. We already run this in production on what's available today. We just figured the people who built the layer it all stands on should be the first to see what got built on top of it.

Already running

Not a proposal or a feature request — a working system. Netmaker is load-bearing in it right now.

You built the foundation

It felt wrong to put your mesh this deep in the core and never show you what it's become.

Just take a look

If any of it is interesting, we'll happily go deeper in the meeting. If not — no harm, no homework.

Netmaker connects the machines. GhostBridge proves **who they are and **that policy was enforced** — every hop, auditor-grade, in an AI loop you can cross-examine.**

You're already in the core of what we built. We'd just like you to be the first to see it — and tell us, straight, what you think.